

Number: P3372R3
Date: 2025-02-11
Audience: LEWG
Reply-to: [Hana.Dusíková](#)

Table of contents

- [Revision history](#)
- [Introduction and motivation](#)
- [What is not included](#)
 - [std::hash](#)
 - [std::hive](#)
 - [non-transient allocations](#)
- [Implementation experience](#)
 - [std::hash](#)
 - [std::less and friends](#)
 - [key & node_handle::key\(\)](#)
 - [libstdc++](#)
- [Impact on existing code](#)
- [Notes for wording changes](#)
- [Proposed changes to wording](#)
 - [Feature test macros](#)

constexpr containers and adaptors

This paper proposes marking all containers and adaptors `constexpr` (with exception of `std::hive`). This will make compile time programming much easier by making normal C++ and `constexpr` C++ more similar.

This paper doesn't propose changes in `std::hash`.

Revision history

- [R2](#) → [R3](#): updated features test macros
- [R1](#) → [R2](#): Removal of `constexpr` for `node_handle::mapped()` member function, Removal of `ceilf` mention in the implementation section as the function is `constexpr` since C++23. Added discussion about hashing and why it's not included. Added informations about `libstdc++` implementability. Extended discussion about change impact on existing code.
- [R0](#) → [R1](#): Added implementation specific problems with `std::less<T*>`.

Introduction and motivation

All necessary `constexpr` functionality required by the containers to be changed is supported by the language. There is no reason for them not to be `constexpr`, and the standard library should support these in order for them to properly serve as vocabulary types.

The following table shows what this paper proposes, and the status quo:

container / adapter	before	after
<code>std::array</code>	✓	✓